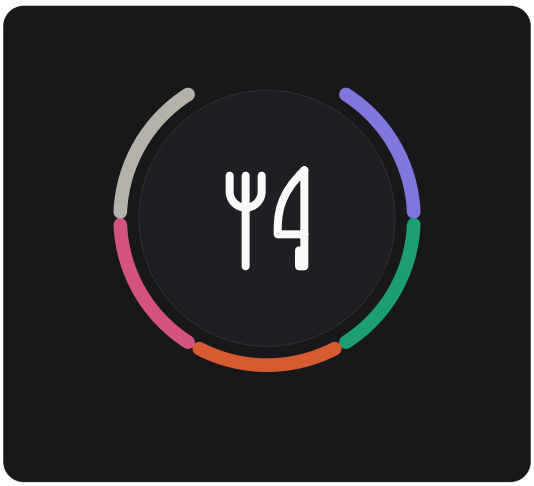

Mise

User Manual

An AI-assisted weekly meal planner
built with Vaadin Flow and Spring AI

Built with Vaadin and Spring AI

https://github.com/petrixh/mise-demo
This manual covers running and using the Mise demo.
For architecture and specifications, see the spec tree in the repository.

Contents

1 Welcome — what is Mise?

Mise is a weekly dinner planner with one defining idea: **a week of dinners is always already there**. You never start from a blank page — the app seeds a full week (plus several weeks of history) the moment a household exists, and from then on you *reshape* the plan by talking to it: "make Thursday vegetarian", "get this week under €80", "should I bother with Lido this week?".

It is also a demo. Mise exists to show how deeply an AI assistant can be woven into a regular business application (Vaadin Flow + Spring AI): the same chat controls data, navigation, charts and grids, and everything the AI changes is visible, explainable, and undoable.

1.1 The three views

- **Plan** — the week's seven dinners with cost, prep time, and calories, plus weekly totals. Edit by chat, pin meals you're hosting, undo anything, navigate between weeks.
- **Shopping** — a consolidated shopping list for the viewed week, grouped by aisle, priced against store catalogs, minus what your pantry already covers. Includes a store recommendation with detour-versus-savings reasoning.
- **Reports** — cost and nutrition trends over the accumulated weeks. The charts and grids themselves can be reshaped by chat, and those customizations persist.

A shared **chat dock** spans all three views: one conversation, aware of which view you're on, persisted across restarts.

1.2 What Mise is honest about

> This is a single-household local demo: **no login, no accounts**. Recipe, store, and persona catalogs are **static seed files** (18 recipes, 3 stores) read from disk at startup — there are no live price feeds, and nutrition values are seeded estimates, not medical data. The AI never fabricates missing data: if there's no price or no nutrition value, it says so instead of inventing one.

2 Setup & configuration

This chapter assumes you can already run a container (or a Maven project) and covers only what is Mise-specific. The published image is `ghcr.io/petrixh/mise-demo` (multi-arch: x86-64 and ARM64); from a checkout, `./mvnw` starts dev mode and `docker build .` produces the same image locally. No Docker? A runnable JAR bundle ships with every release too (see Section 2.1.2).

2.1 The one mandatory decision: an LLM endpoint

Mise talks to any **OpenAI-compatible** chat-completions endpoint — api.openai.com, Azure, Groq, or a local server such as llama.cpp, llama-swap, LM Studio or Ollama. Configuration is via environment variables:

Variable	Meaning	Default
`MISE_MODEL_BASE_URL`	Base URL of the endpoint. **Must include the `/v1` path segment** — the OpenAI SDK appends `/chat/completions` directly to it.	a LAN demo host
`MISE_MODEL_API_KEY`	API key, if the endpoint needs one.	`not-needed`
`MISE_MODEL_NAME`	Model id as the endpoint knows it.	a local Qwen
`MISE_MODEL_MAX_TOKENS`	Response-length cap per turn.	`16384`
`MISE_MODEL_TIMEOUT`	Per-call timeout for the model — covers both silence between streamed tokens and the whole turn. Raise it if big Reports reshapes abort mid-answer on a slow host (see Troubleshooting). Accepts `180s`, `3m`, `PT3M`.	`180s`
`PORT`	HTTP port the app listens on.	`8080`

2.1.1 Which model? Recommendations for local hosting

Any capable OpenAI-compatible model works, but Mise's chat is unusually tool-heavy, and the project benchmarked a field of local candidates against its AI test suite (22 tool-invocation, grounding, and tone checks — see issue #4 in the repo). Two models passed everything and are the recommended picks:

- **Qwen 3.6 35B-A3B** (`Q4_K_XL` quant) — the reference baseline. 22/22.
- **Qwen 3.5 4B** (`Q8_K_XL` quant) — also 22/22, and the surprise headliner: it matched the 35B's quality while running the suite 30% faster (sub-half-second first token). At a 4 GB weight footprint it fits with usable context on an 8 GB-class GPU — a perfectly good way to run Mise on modest hardware.

One expectation-management note on the small model: passing the test suite means it operates Mise's tools correctly — it doesn't make a 4B model as smart as a 35B one. Off the suite's beaten path (open-ended questions, multi-step negotiations, nuanced "why" explanations) the bigger model is noticeably more capable, so don't judge what Mise can do by the small model's ceiling. The 4B is the budget pick; the 35B is the experience.

One concrete spot the 4B struggles: the most complex Reports reshape — the month-bucketed "vegetarian dinners by month" chart — where it tends to get stuck retrying SQL variations rather than recovering cleanly. Simpler reshapes (the leaderboard examples) and the day-to-day chat work fine on it. If a chart reshape spins, retry or switch to the 35B.

What the bake-off taught us, if you want to substitute your own: at this size, **quantization and instruct discipline matter more than parameter count** (a 9B at default quant scored 59% where the 4B at Q8 scored 100% — prefer Q8 over Q4 for small models); models below 4B and the Llama 3.2 family got stuck in tool-call loops on Mise's multi-turn flows; and Gemma didn't speak the OpenAI tool-call format at all through the hosts we tried.

A complete container run, with persistence (see below):

```
docker run -p 8080:8080 \
  -e MISE_MODEL_BASE_URL=https://api.openai.com/v1 \
  -e MISE_MODEL_API_KEY=sk-... \
  -e MISE_MODEL_NAME=gpt-4o-mini \
  -v mise-data:/data \
  ghcr.io/petrixh/mise-demo:latest
```

Running from a checkout instead? Copy `application-local.properties.example` to `application-local.properties` (project root, gitignored) and set the same values there — the file is auto-loaded for every dev run.

2.1.2 No Docker? Run the JAR directly

The only prerequisite is **Java 21 or newer**. Each release's CI run stores a `mise-demo-<version>-jar` artifact (repo → Actions → the Release run for the tag → Artifacts; downloading needs a GitHub login, and artifacts expire after 90 days). The zip contains the runnable JAR plus the `demo/` seed-catalog directory it reads at startup — extract it and run **from the extracted directory**:

```
MISE_MODEL_BASE_URL=https://api.openai.com/v1 \
MISE_MODEL_API_KEY=sk-... \
MISE_MODEL_NAME=gpt-4o-mini \
java -jar mise-demo-<version>.jar
```

The same environment variables as the table above apply. Data persists to `./data` next to the JAR (see below), and a JAR built without a Vaadin license shows a watermark banner — expected, not a bug.

2.2 Persistence — what survives a restart

Mise stores its state (household, plans, pantry, preferences, report customizations, and the full chat history) in an H2 file database. In the container it lives at `/data/mise`; mount a volume at `/data` and everything survives container restarts:

```
-v mise-data:/data        # named volume, or -v /some/host/dir:/data
```

Factory reset: stop the container and delete the volume (`docker volume rm mise-data`) — on the next start Mise is back to first-run state. Running the JAR directly (or from a checkout), the database lives in `./data` relative to the working directory instead: re-running from the same directory resumes your data, and deleting `data/` is the reset.

2.3 First run: the onboarding interview (and what personas add)

On first run Mise has no household yet, so it opens with a short onboarding chat — household size, things you can't or won't eat, a rough weekly budget. **Your answers are what define the household**: size, budget, allergies, dislikes, and hosting pattern all come from that conversation, and the weekly plan is built from them (see Section 3.1).

A **persona** file supplies the backdrop around your answers: the starting pantry staples, cuisine preferences, a fallback for loved foods if you don't mention any, and how many weeks of plan history to seed (so Reports has trends from day one). The single line in `demo/data/active_persona.txt` picks which persona is used — `helsinki-family` or `young-couple` out of the box. In the container these files sit at `/app/demo/data/`; mount your own directory over that path to change them.

Both the interview and the persona apply **on first run only** — afterwards the household lives in the database. Wipe it (factory reset above) to go through onboarding again.

2.4 The H2 console, for the curious

The running app exposes its database at `http://localhost:8080/h2-console` (also linked from the side drawer). Connect with JDBC URL `jdbc:h2:file:/data/mise;AUTO_SERVER=TRUE;DB_CLOSE_DELAY=-1` (from a checkout: `./data/mise` instead of `/data/mise`), user `sa`, empty password. It is a demo-grade database — don't expose the port beyond your machine.

3 Using Mise

3.1 First launch

On first launch Mise greets you in chat and asks a couple of questions — household size, things you can't or won't eat, a rough weekly budget. Answer in plain language ("Two adults, no fish for me, around €90 a week"); within a few turns the household is created from exactly what you said, and you land on the Plan view with a full week of dinners already in place. Several weeks of history are seeded too (from the persona's `seedWeeks`), which is why Reports has trends to show from day one.

3.2 The Plan view

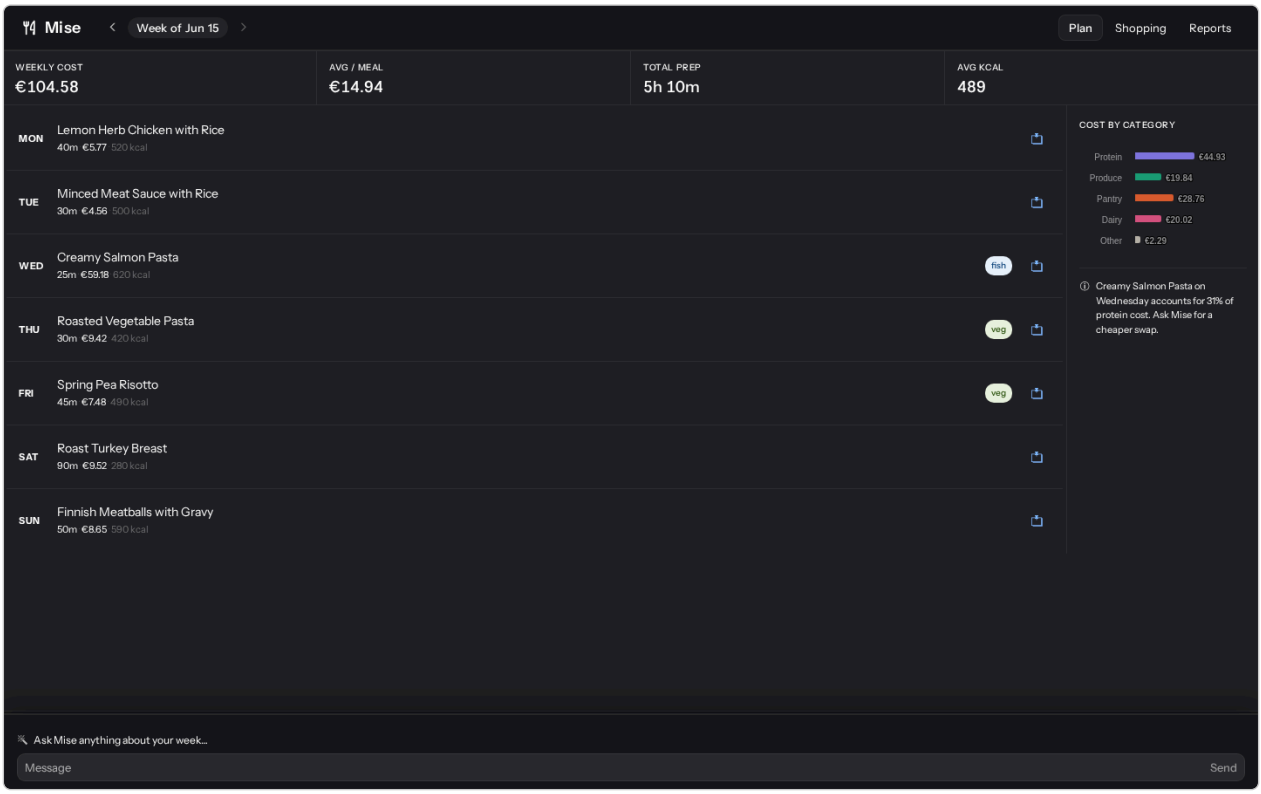

Figure 1: The Plan view: the week's dinners, weekly totals, and the chat dock.

The grid shows Monday through Sunday, each row a dinner with its category, prep time, estimated cost, and calories. Above it, a summary strip totals the week. Things you can do:

- **Edit by chat.** "Make Wednesday vegetarian", "swap Monday for something faster", "get this week under €80 without dropping Sunday's cod". The AI explains what it changed and why; changed rows are marked as AI-edited.
- **Pin a meal** you don't want touched — click the pin on the row or say "pin Saturday, I'm hosting". Pinned meals are skipped by later edits.
- **Undo.** Every AI edit is reversible: use the undo affordance on a recently edited row, or just say "put Wednesday's old meal back".
- **Ask why.** "Why did you change that?" gets an explanation grounded in your actual preferences, prices, and prep times — and if no reason is on record, Mise says so rather than inventing one.
- **Navigate weeks** with the chevrons or the date picker in the header. The viewed week is independent of the current week — you can review history or look at planned future weeks, and the chat answers about whichever week you are viewing (ask "what's on Wednesday?" on a past week and you get that week's Wednesday).
- **Plan ahead.** "Plan next week" (or "plan June") generates future weeks that respect the same constraints; the next-week chevron lights up when one exists.

3.3 The Shopping view

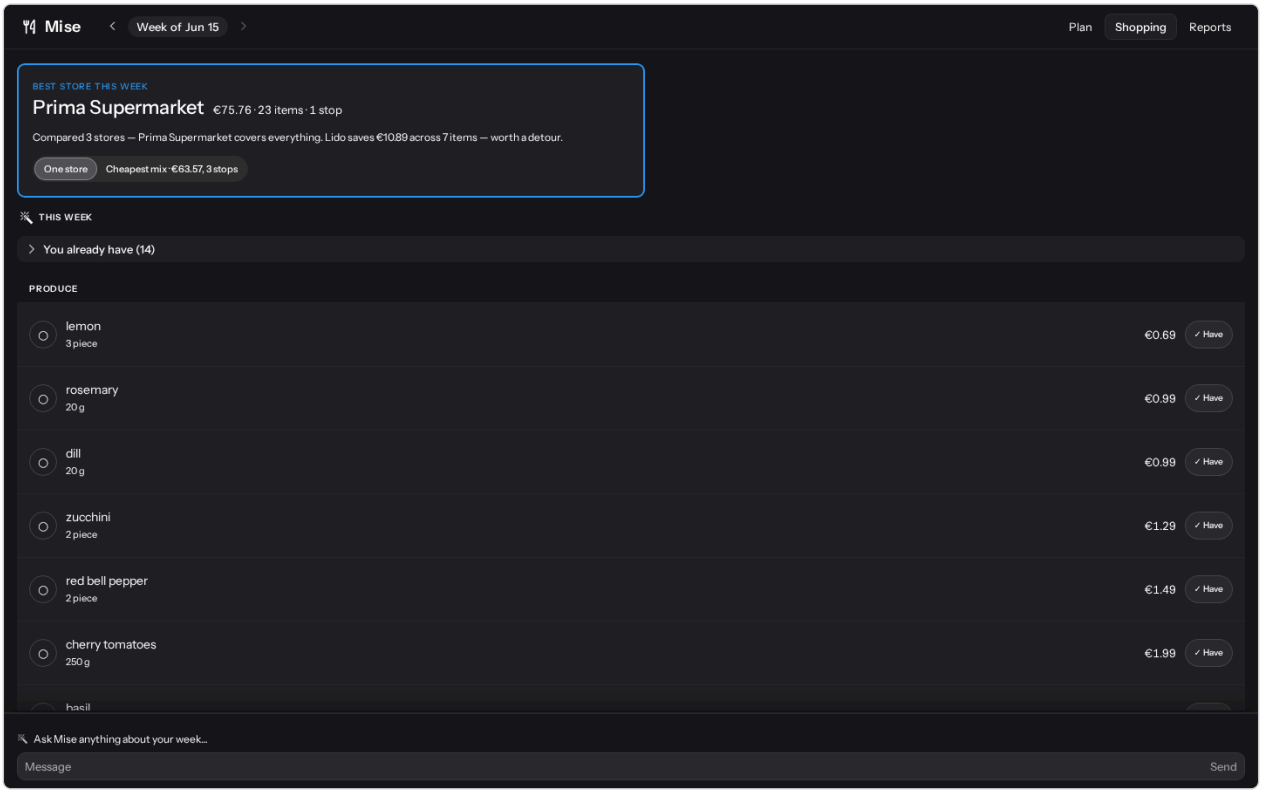

Figure 2: The Shopping view: aisle-grouped list, store strip, and the One store / Cheapest mix toggle.

Shopping shows everything the viewed week needs, consolidated across recipes (two recipes wanting carrots become one line), grouped by aisle, and priced. Items your pantry already covers are subtracted into a separate "you already have" section.

- **Store modes.** The toggle switches between *One store* — everything from the single cheapest store for the whole basket — and *Cheapest mix* — items split across stores for the lowest total, with per-item store labels.
- **Store recommendation.** The header strip names the recommended store and the weekly total. Ask "should I bother with Lidio this week?" and Mise weighs the actual savings against the detour and answers with euro amounts, naming the items that drive the difference.
- **Savings without the detour:** say so — "I want the savings without the second stop" — and Mise proposes a meal swap that shifts ingredients toward the store you're already visiting.
- **Check items off** as you shop; tap an item to mark it as already-in-pantry. Check-off state is for the current trip and resets when the plan changes.

3.4 The Reports view

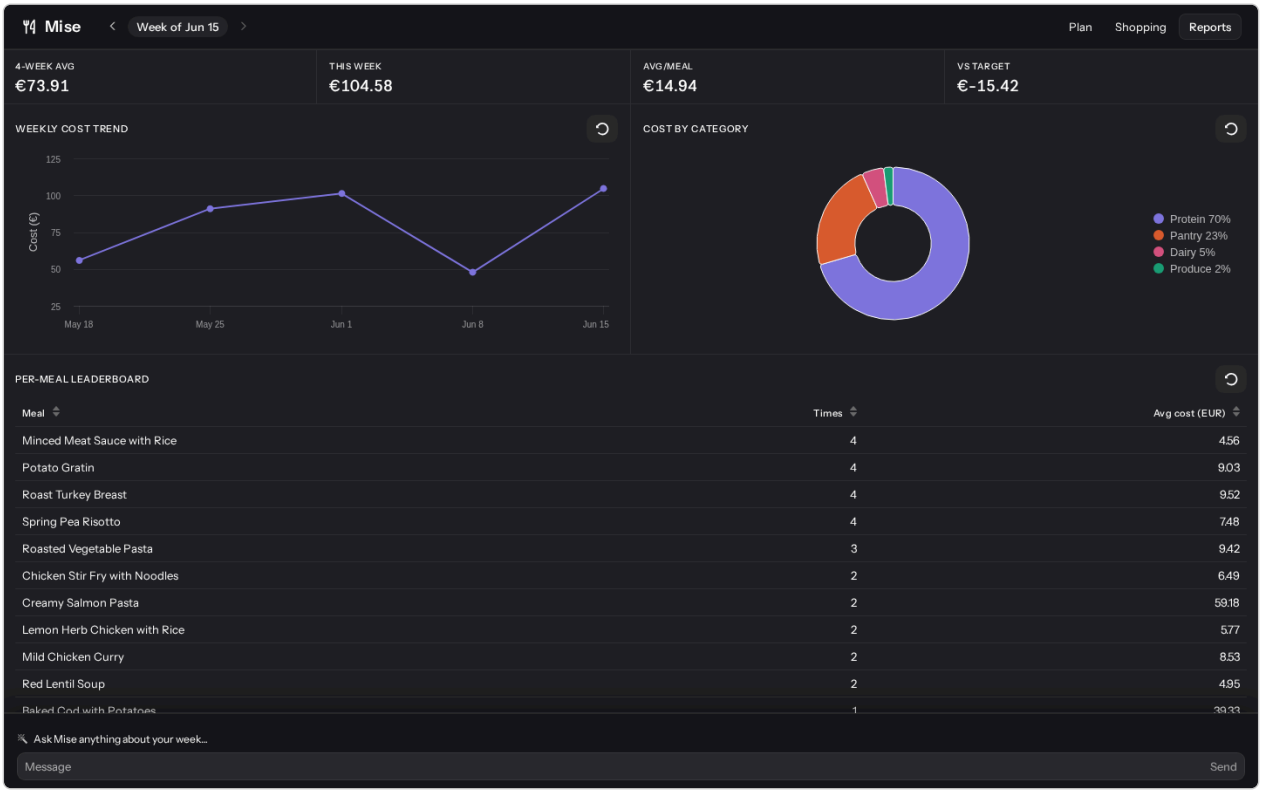

Figure 3: The Reports view: trend chart (here already repointed by chat to “vegetarian dinners by month”), category breakdown, and the recipe leaderboard.

Reports charts the accumulated weeks: a weekly cost trend, a cost-by-category breakdown, and a recipe leaderboard. What makes it unusual is that the widgets themselves answer to chat:

- **Repoint a chart:** “show me a chart of how often I cook vegetarian dinners by month” — the trend chart switches to exactly that.
- **Reshape a grid:** “in the leaderboard, rank by kcal per euro”.
- **Ask questions:** “why was last week more expensive?” — answered from the actual data, citing the meals and categories responsible.

Customizations **persist**: reload the page or restart the app and your reshaped widgets are still there. To go back, use the reset affordance on a widget’s header or say “reset the leaderboard”.

3.5 The chat dock, insights, and trust

The chat dock follows you across all three views as **one** conversation — you can stand on the Plan view and say “go to reports and add a kcal column”, and both things happen. Mise knows which view (and which week) you’re looking at, so “what’s on Friday?” means Friday of the viewed week. The conversation itself is persisted; after a restart you can still ask “what did I ask earlier?”.

From time to time Mise volunteers an **insight** (“next week is trending €15 over budget”). Insights are advisory: they change nothing on their own, and you can dismiss them or follow up in chat.

The general contract: **everything the AI changes is inspectable and reversible.** Edits are marked, explanations are grounded in real data, undo is always available, and when data is missing the AI declines instead of guessing.

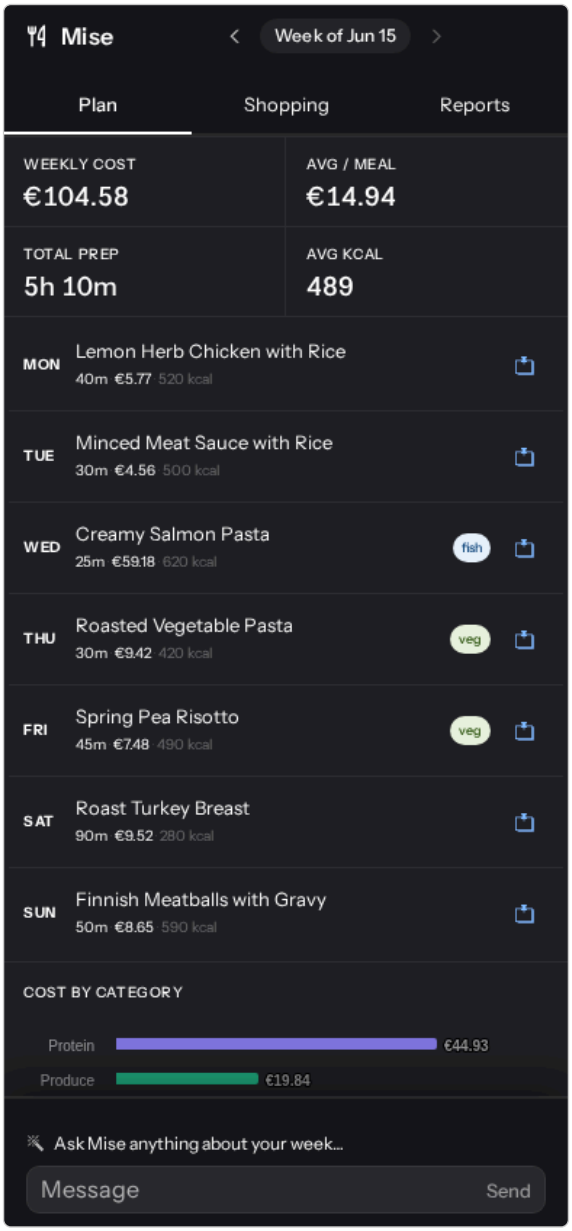

Figure 4: Mise on a phone: the views stack and the chat dock docks to the bottom.

4 Example queries

All of the following were verified against a running build of this version of Mise. "View" is where the effect lands — remember you can issue any of them from anywhere.

You say	View	What happens
"Make Wednesday vegetarian, kid is having a friend over."	Plan	Wednesday's dinner is replaced by a vegetarian recipe; the row is marked as an AI edit; weekly totals and the shopping list update. (Asked while Wednesday is already vegetarian, Mise says so and changes nothing.)
"Why did you change that?"	Plan	A short, grounded explanation of the most recent edit — e.g. "I swapped Spaghetti Bolognese to Roasted Vegetable Pasta because you asked to make Wednesday vegetarian". No fabricated reasons.
"Put Wednesday's old meal back."	Plan	The previous Wednesday meal is restored (undo via chat).
"Pin Saturday — I'm hosting."	Plan	Saturday is locked; later edits and replans leave it alone.
"Get this week under €80 without dropping Sunday's cod."	Plan	Meals are swapped to bring the total under €80, respecting the named meal *and* any pins. In our verified run: Monday's roast became Red Lentil Soup, the week landed at €79.61, cod and pinned Saturday untouched.
"What's on Wednesday?" (while viewing an earlier week)	Plan	Date questions answer for the week you are currently **viewing**, not today's week. Navigated to the week of June 1, this returned "Wednesday's dinner is Spring Pea Risotto on Wednesday, June 3rd" — the viewed week's Wednesday, not the current week's. Step back to today's week via the chevrons or the Mise wordmark and the same question answers for this week instead.
"Plan next week."	Plan	A new week is generated for the Monday after your current week — 7 dinners honoring the same allergy / budget constraints — and the next-week chevron lights up so you can navigate into it. In our verified run: "Next week (June 22–28) is planned — 7 dinners, estimated €78."
"Plan the rest of June."	Plan	Mise resolves the relative range and generates every remaining Monday that isn't already on your calendar, skipping the ones that are. In our verified run: "I've planned the week of June 29 (7 dinners, est. €54). The week of June 22 was already planned."
"Should I bother with Lido this week?"	Shopping	A verdict with euro amounts: the savings at Lido versus the detour, naming the items that drive the difference ("€2.40 across 5 items, but the detour adds 8 minutes — not worth the trip").
"I want the savings without the detour."	Shopping	Mise proposes concrete meal swaps that avoid the detour store, with the saving per swap, and asks before applying any of them.

You say	View	What happens
“Why was last week more expensive?”	Reports	An answer computed from your data, citing the meals/categories that drove the difference (in our run it named one €59 salmon dinner as the culprit).
“Show me a chart of how often I cook vegetarian dinners by month.”	Reports	The trend chart is repointed to that data — and the new shape persists across restarts.
“In the leaderboard, rank by kcal per euro.”	Reports	The leaderboard grid is reshaped with the computed column/ordering.
“For the leaderboard, show one row per week with columns for fish, meat, veg and chicken meals, plus the average kcal per euro that week.”	Reports	A fuller reshape: the grid pivots to a week-per-row table with category-count columns and a computed value column — all from one sentence. (A heavy turn on a slow model; if it aborts mid-answer, raise `MISE_MODEL_TIMEOUT` — see Troubleshooting.)
“Reset the leaderboard.”	Reports	The widget returns to its default shape.
“Chart my carbon footprint per meal.”	Reports	**Graceful failure:** Mise declines — there is no carbon data — and offers the nearest real proxy (e.g. cost or calorie intensity) instead of inventing numbers.

5 Make it yours — extending the seed data

The catalogs Mise plans from are plain files under `demo/data/` (in the container: `/app/demo/data/`, overridable with a volume mount):

```
demo/data/
├── active_persona.txt      # one line: which persona file is active
├── personas/*.json         # onboarding backdrop (pantry, cuisine prefs, history)
├── recipes/*.yaml          # the recipe catalog
└── stores/*.yaml           # store price catalogs
```

Recipes and stores are read **fresh at every startup** and are never copied into the database — edit a file, restart, and the change is live. Personas are the exception: they are consumed once, on first run (see Section 5.3).

5.1 Adding a recipe

Drop a new `.yaml` into `demo/data/recipes/` and restart. A complete, working example:

```yaml
# demo/data/recipes/halloumi-traybake.yaml
id: halloumi-traybake        # optional — defaults to the filename stem
name: "Halloumi Traybake"
cuisine: Mediterranean
categoryTags: [vegetarian, weeknight, easy]   # free-form; drives "make it
vegetarian" etc.
prepMinutes: 35
defaultServings: 4
estimatedCost: 12.00         # fallback only — real cost comes from store prices
ingredients:
  - name: halloumi           # must match a store item — see the gotcha below
    quantity: 450
    unit: g
    aisle: dairy             # drives shopping-list grouping — reuse existing names
    optional: false
  - name: bell pepper
    quantity: 3
    unit: piece
    aisle: produce
    optional: false
  - name: olive oil          # staple — pantry usually covers it
    quantity: 30
    unit: ml
    aisle: dry-goods
    optional: false
  - name: chili flakes
    quantity: 5
    unit: g
    aisle: dry-goods
    optional: true           # optional: skipped by allergy checks, droppable
macros:                      # per serving, seeded estimates
  kcal: 520
  protein: 24
  carb: 38
  fat: 30
notes: "Roast hot; add the halloumi for the last 10 minutes."
```

The two gotchas that make or break a custom recipe:

1. **Every `ingredients[].name` must match a `ingredientName` in a store catalog** (matching is case-insensitive and tolerant of substrings — e.g. "chicken breast" matches a "chicken breast" store item). If nothing matches, the item has **no price**: it still appears on the shopping list, but the recipe's and week's costs display as degraded/unknown. After adding a recipe, check the Shopping view for price-less lines.
2. **`aisle` values drive shopping-list grouping.** Reuse the names already in use — `produce`, `meat`, `fish`, `dairy`, `dry-goods`, `frozen`, `beverages` — or your items land in a lonely group of their own.

5.2 Adding or editing stores

Store files pair a price catalog with the detour metadata the recommendation logic reasons about:

```yaml
# demo/data/stores/lido.yaml (excerpt)
id: lido
name: "Lido"
detourMinutesFromRoute: 12   # what "should I bother with Lido?" actually reads
defaultStore: false          # exactly ONE store must have true (Prima, by default)
catalog:
  - ingredientName: chicken breast
    price: 3.49
    unit: kg
    packSize: 1
    onSale: true
    saleUntil: 2026-06-30
```

Cheapen an item in a non-default store and restart — the detour reasoning and the *Cheapest mix* toggle pick it up immediately. This is the intended demo move: change the data, watch the AI's recommendation change.

5.3 Personas — the onboarding backdrop

To plan for a different family you don't need to touch any file — wipe the database and answer the onboarding interview differently. Edit a persona only to change the backdrop the interview doesn't cover: the staple pantry (`defaultPantry`), `cuisinePrefs`, and how many weeks of history get seeded (`seedWeeks`). Adjust a copy of `helsinki-family.json`, point `active_persona.txt` at the new id, and wipe the database — personas are read only when onboarding runs.

5.4 Restart or reset?

Change	Needs
Add or edit a recipe YAML	restart
Add or edit a store / prices / sales / detour minutes	restart
Change `active_persona.txt`	DB wipe + restart
Edit a persona JSON	DB wipe + restart
Anything created in-app (plans, pantry, report widgets, chat)	already persistent

Note that history already snapshotted into the database keeps its old values when seed files change — past weeks don't rewrite themselves because a price moved.

6 Troubleshooting & FAQ

The chat doesn't answer (or errors). Almost always the endpoint config. Check, in order: `MISE_MODEL_BASE_URL` **includes `/v1`** (the classic miss); the endpoint is reachable *from inside the container*; `MISE_MODEL_API_KEY` and `MISE_MODEL_NAME` are what your provider expects.

A big Reports reshape spins, then the answer cuts off. On a slow or loaded model the request can outlast the default per-call timeout (60s out of the box) and get dropped mid-stream — the widget never updates. Raise `MISE_MODEL_TIMEOUT` (it ships at `180s`; try `300s` for CPU-only hosts). This covers both long silences between streamed tokens and the overall length of a single turn. A faster or less-loaded model is the other lever.

All my data disappeared after a container restart. No volume was mounted at `/data`. Mount one (`-v mise-data:/data`) — the database can't survive the container's filesystem otherwise.

I edited a persona but nothing changed. Personas seed the household on first run only. Wipe the database (delete the volume / `data/` directory) and restart.

A shopping item has no price / weekly cost looks wrong. A recipe ingredient name doesn't match any store catalog item. Add the ingredient to a store YAML (or align the spelling) and restart.

Charts show a license watermark. The image was built without a Vaadin license (the chart component is commercial). Functionality is unaffected; builds with a license key don't show it.

Where do I file issues? github.com/petrixh/mise-demo/issues.